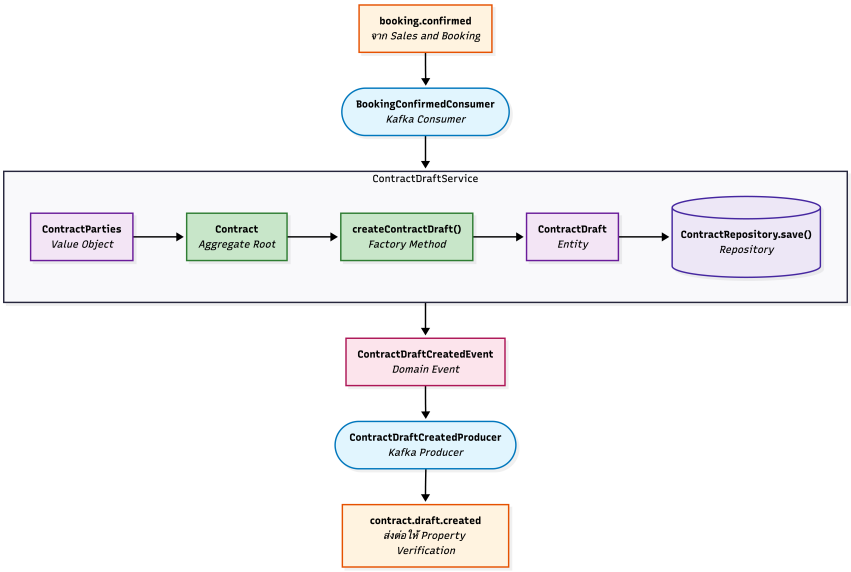
1. Microservice

Domain Model Design ของทีม Contract & Legal แบ่งออกเป็น 2 Sub-domain ในเอกสารนี้จะแสดงการนำ Sub-domain 1 Contract Creation มา implement เป็น microservice โดยเลือก 1 event flow คือ ContractDraftCreated

2. DDD Tactical Patterns ที่เลือกใช้

Pattern	สิ่งที่ implement	เหตุผลที่เลือก
Aggregate Root	Contract	เป็นจุดเข้าถึงเดียวของ aggregate ควบคุม lifecycle + consistency ของ ContractDraft ภายใน ใช้ @Version สำหรับ optimistic locking
Entity	ContractDraft	มี lifecycle ของตัวเอง (สร้าง/แก้ไขได้ก่อน finalize) แต่ต้อง access ผ่าน Contract Root เท่านั้น map เป็น @OneToOne ภายใน aggregate
Value Object	ContractStatus, ContractTerms, ContractParties	ไม่มี identity กำหนดค่าครั้งเดียว immutable ใช้ enum และ @Embeddable ของ JPA
Domain Event	ContractDraftCreatedEvent	ใช้ past tense ตาม DDD convention เป็น immutable record ที่ส่งผ่าน Kafka เชื่อมระหว่าง sub-domain
Factory Method	Contract.createContractDraft()	encapsulate business logic การสร้าง aggregate ไว้ใน Root ไม่ให้ภายนอกสร้าง entity โดยตรง
Repository	ContractRepository	abstraction layer สำหรับ persistence ใช้ Spring Data JPA

3. Event Flow ที่ implement



4. Aggregate Design

Contract (Root) เก็บแค่ ID อ้างอิงไปยัง domain อื่น (bookingId, customerId, unitId) ไม่เก็บข้อมูลซ้ำ เพื่อหลีกเลี่ยง inconsistency เมื่อ domain ต้นทางเปลี่ยนแปลง

ContractDraft (Entity) ถูกสร้างผ่าน factory method ของ Contract Root เท่านั้น ภายนอก aggregate ไม่สามารถสร้างหรือแก้ไขได้โดยตรง

Value Objects ใช้ @Embeddable ฝังอยู่ใน Contract table โดยตรง ไม่ต้องสร้าง table แยก เพราะไม่มี identity และผูกกับ aggregate 1:1

5. Kafka Event Design

Consume: booking.confirmed

Field	Type	คำอธิบาย
bookingId	string (uuid)	ID ของ booking ที่ยืนยันแล้ว
unitId	string (uuid)	unit ที่ลูกค้าจอง
customerId	string (uuid)	ลูกค้าผู้ซื้อ

Produce: contract.draft.created

Field	Type	คำอธิบาย
contractId	uuid	ID สัญญาที่ระบบสร้างขึ้น
bookingId	uuid	อ้างอิง booking ต้นทาง
unitId	uuid	unit ที่ทำสัญญา
customerId	uuid	ลูกค้าผู้ซื้อ
status	enum	"DRAFT"
fileUrl	string	URL ไฟล์ร่างสัญญา PDF
templateId	uuid	template ที่ใช้สร้างร่าง
createdAt	ISO8601	เวลาสร้าง aggregate
draftedAt	ISO8601	เวลาสร้างร่างสัญญา

6. วิธีรันระบบ

```
# 1. Start Kafka + Spring Boot

docker-compose up --build

# 2. ทดสอบ

curl -X POST http://localhost:8081/api/test/booking-confirmed
```

ดูผลลัพธ์ใน log ของ docker-compose และ H2 Console (http://localhost:8081/h2-console, JDBC URL: `jdbc:h2:mem:contractdb`)

โดยสามารถดูรายละเอียดการ run โค้ดเพิ่มเติมได้ใน README.md